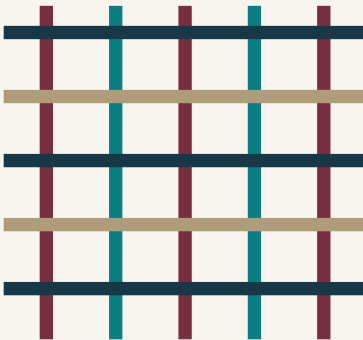




WITHAQ

Concept, Architecture and Build-to-Run Guide

Function-aware device containment, governed AI disclosure, and data revocation with workflow recovery.

EDITION 2.0 | 28 SEPTEMBER 2026

PROJECT STATUS: PROPOSED DESIGN; PROTOTYPE INCOMPLETE

WITHAQ PROJECT TEAM | DESIGN, BUILD AND EVALUATION

Reader guide and contents

Read the abstract and scenario for the concept, contracts and algorithm for containment, then revocation, dispatch and implementation interfaces. Identifiers and figures match across editions. Operational figures are lab targets unless explicitly labeled as an illustrative computation.

01	WITHAQ: concept and reading map	5
02	Problem statement and objectives	6
03	Project scope and limitations	7
04	End-to-end reference scenario	8
05	Logical architecture and trust boundaries	9
06	Runtime structure and technology stack	10
07	Data flow diagrams	11
08	Use case model	12
09	Actors and user stories	13
10	Functional requirements: devices and containment	14
11	Functional requirements: data and integration	15
12	Non-functional requirements	16
13	Function contracts and dependency graphs	17
14	MFSC algorithm and decision rules	18
15	Worked decision example and boundaries	19
16	Enforcement, validation and recovery	20
17	Generative AI disclosure guard	21
18	MAHW: revocation, lineage and recovery	22
19	Concurrency and dispatch guarantee boundaries	23
20	System class diagram	24

Contents | continued

21	Containment sequence diagram	25
22	Revocation and recovery sequence diagram	26
23	Entity-relationship model: network	27
24	Entity-relationship model: data	28
25	Entity-relationship model: dispatch and evidence	29
26	API and service contracts	30
27	Zero-to-start: host and tools	31
28	Repository creation and dependency locking	32
29	First runnable service	33
30	Configuration and local database	34
31	Compose, migrations and readiness	35
32	Implementing the authoritative state model	36
33	Building planner, validator and worker	37
34	Building MAHW and the disclosure gateway	38
35	Building and connecting the console	39
36	Local startup in the correct order	40
37	Real network enforcement laboratory	41
38	Lab deployment, networks and credentials	42
39	Complete demonstration and expected evidence	43
40	Acceptance tests and traceability	44

Contents | continued

41	Operations, observability and failure recovery	45
42	Stop, resume and backup procedures	46
43	Troubleshooting common failures	47
44	Complete MVP file-tree contract	48
45	File inventory: Root and configuration	49
46	File inventory: Application core and routes	50
47	File inventory: Domain models and schemas	51
48	File inventory: Decision and guard services	52
49	File inventory: Persistence and migrations	53
50	File inventory: Adapters and separate processes	54
51	File inventory: User interface	55
52	File inventory: Lab data and policy	57
53	File inventory: Tests and engineering records	58
54	Delivery order and definition of done	59
55	Glossary	60
56	References and implementation sources	61
57	Official setup links and edition control	62

Figures

1 Architecture; 2 Data flow; 3 Use cases; 4 Classes; 5-6 Sequences; 7-9 Entity relationships. Vector artwork remains sharp when printed or enlarged.

WITHAQ: concept and reading map

WITHAQ is the current name of the project previously developed as SIYAJ, with MAHW as an internal layer. Imagine a device gateway collecting monitoring readings, an application producing a report, and a user requesting that report be sent to an AI service. If the gateway or its data becomes untrusted, WITHAQ determines what must be restricted, what can continue, and what must no longer be used or disclosed.

Layer	Plain-language explanation
Device containment	Restrict a suspect device’s connections while measuring each feasible critical function.
Disclosure guard	Check data, destination, account and purpose before managed AI disclosure.
MAHW	Revoke use of a source and its derivatives, then rebuild from a permitted alternative where available.

What this report delivers

This is a build and operations blueprint, not a completed application repository. It specifies files, interface contracts, setup examples and verification steps. WITHAQ service commands become runnable after their listed modules are implemented; examples do not imply that the project prototype has passed tests.

Start with the concept, scenario and architecture, then read the contracts and diagrams. Follow the build gates in order and use the final file inventory to assign work. Identifiers and commands match across editions; code names remain untranslated for implementation. Baseline status: proposed design, prototype incomplete according to the latest team information.

Problem statement and objectives

When trust in a device or data source changes, an organization must choose a response that enforces security constraints, preserves each feasible critical function and prevents subsequent prohibited data use, with auditable evidence. Full isolation may interrupt needed work; unrestricted continuation retains risk; superficial deletion leaves derivative dependencies active.

Objective	Proposed verification criterion
O1	Version assets, attack paths and contracts with provenance evidence; detect an injected unknown dependency.
O2	Select enforceable restrictions from a bounded set, blocking every modeled attack path while satisfying every critical contract.
O3	Inspect every request on the managed dispatch path; reject prohibited disclosure and detect a gateway bypass attempt.
O4	Reject reads or publication of derivatives of a revoked root after the revocation barrier, including late results.
O5	Rebuild a new output from an approved alternative while an independently proven branch continues.
O6	Compare against baselines and report failure, uncertainty and functional loss, not just average performance.

Integrated success criterion

Success requires confirmed enforcement, measured satisfaction of every critical contract, and current use and disclosure guards. One layer may succeed while another fails; the interface must show that explicitly.

Project scope and limitations

Within the prototype	Outside current evidence
Linux lab network, device gateway, monitoring and alert services, protected administrative target.	Medical-device certification, therapeutic control or live hospital deployment.
8-12 defined deny actions; one network enforcement point initially.	Optimality over large networks or an unrestricted action space.
Managed enterprise AI workflow with synthetic Arabic and English data.	Visibility into all personal use, unintegrated apps or encrypted bypass channels.
Application lineage, logical revocation and recovery from permitted inputs.	External model unlearning, recall of already transmitted bytes or guaranteed physical backup erasure.

The model assumes covered traffic crosses the enforcement point and that policy authority, state storage and service credentials are uncompromised. Losing these assumptions limits the evidence and cannot be solved by adding a dashboard. Reading integrity is assessed against a known laboratory generator; connectivity is not evidence of truthful data.

Compliance and data residency require later deployment-specific institutional review. This report does not claim certification by an authority or university. Its academic presentation uses author-date references without implying affiliation with Harvard University.

End-to-end reference scenario

A synthetic generator sends one sample per second through gateway G to monitor M and alert service A. Archive L and protected administration P are also present. On suspicion, G must lose access to P and direct Internet destinations while monitoring and alerts may continue if those flows are separable and verifiable. The reporting application receives time-bounded batches through its approved path.

Step	WITHAQ behavior and visible result
1	Link incident I1 to G with time-specific evidence; MFSC proposes a plan and the validator rechecks it.
2	The enforcement point installs restrictions; an independent probe checks denial to P and contracts for M and A.
3	Suspect batch S1 is held for review; the data steward decides revocation or retention from evidence.
4	A user requests a summary dispatch; the gateway checks content, destination and purpose. Held or revoked roots cannot leave.
5	Revoking S1 makes MAHW block dependent T1 and T2; T3 derived from S3 continues after independence is verified.
6	Rebuild T2 as new object T2b from approved S2; validate it before publication. T1 remains blocked if no alternative exists.

An incident does not automatically revoke the device’s entire history. A device-to-source association records a time window, confidence and rationale, while unknown batches remain explicit. MAHW does not automatically delete forensic evidence; it is retained in a separate store under distinct access and retention purposes.

Logical architecture and trust boundaries

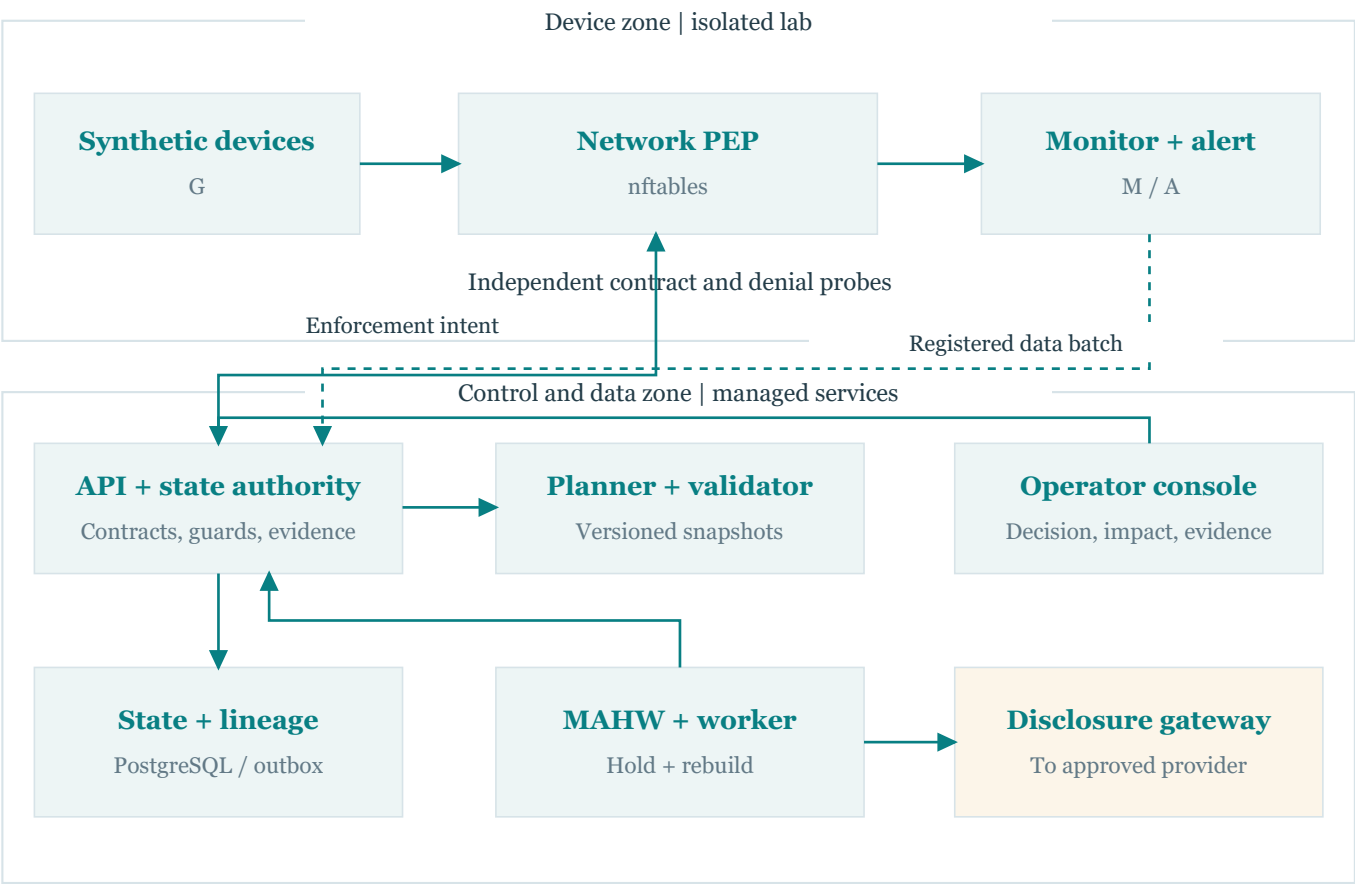


Figure 1. WITHAQ architecture: device and data paths with three enforcement authorities. Arrows show logical exchanges; deployment policy constrains actual network access.

The console obtains state through the API. The planner constructs a plan from a fixed snapshot and an independent validator recomputes its conditions. The API stores authoritative state and versions and emits execution intents through an outbox. A restricted worker performs long jobs; only the network adapter can change lab enforcement rules.

Necessary device traffic crosses a network policy enforcement point (PEP), derivative reads and publication cross an application guard, and external requests cross the dispatch gateway. These authorities share policy and incident versions but maintain separate confirmation evidence. No LLM is authoritative in a binding security decision.

Runtime structure and technology stack

Build the MVP as a modular backend with separate execution, validation and dispatch processes. The API owns state; the worker claims jobs and submits results through internal endpoints without database credentials. The validator consumes snapshots, the gateway owns provider credentials, and only the PEP agent has network privileges inside the lab.

Component	Implementation choice	Rationale
Console	React + TypeScript + Vite	Arabic/English state, contract and lineage views.
API	Python + FastAPI + Pydantic	Typed schemas, OpenAPI and centralized guards.
State	PostgreSQL + SQLAlchemy + Alembic	Transactions, row locks and migrations; one authority.
Planning	NetworkX + bounded enumeration	Explain candidates within 8-12 actions.
Jobs	PostgreSQL outbox + worker	Commit work with its event and retry under guards.
Network	Linux VM + namespaces + nftables	Measurable enforcement away from the host network.
Validation	pytest + Playwright + probes	Contract, race and user-journey testing.

A GPU is unnecessary for the planner, database or mock model. Start retrieval with explicit source lists rather than a separate vector database; add pgvector after proving lineage and guards. Language-model opinions never authorize blocking decisions. Pin dependency versions and hashes after the first compatibility run rather than labeling untested dependencies production-ready.

Data flow diagrams

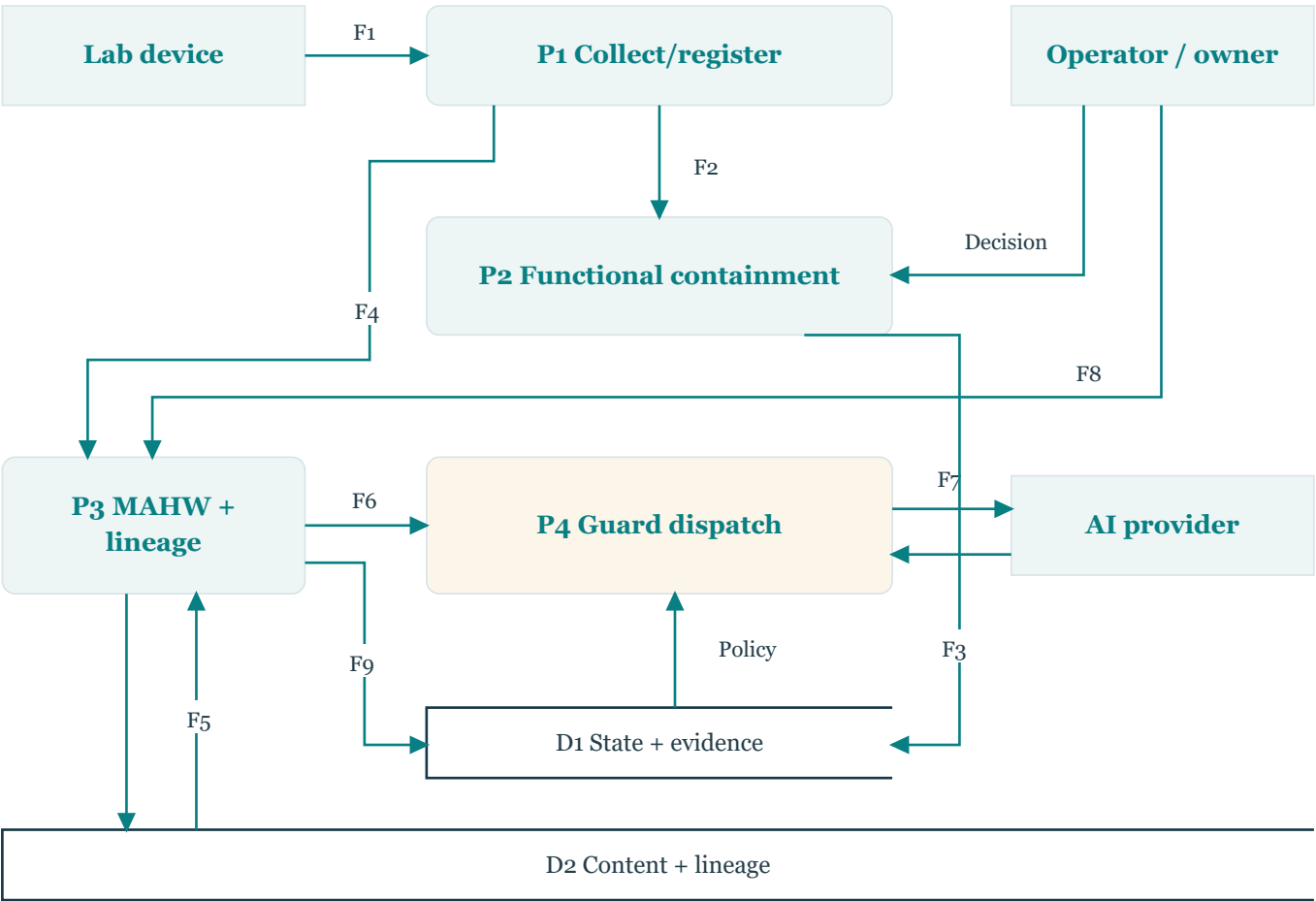


Figure 2. Level-1 data flow diagram. Rectangles are external entities, rounded boxes are processes and open-sided symbols are stores; F labels refer to the flow dictionary.

Flow	Content and control
F1	Measurement, device ID and time; synthetic data, not automatic trust evidence.
F2 / F3	Alert and observations to response; graph snapshot, contracts, plan and evidence in D1.
F4 / F5	Registered source batch, then guarded reads/writes of artifacts and lineage in D2.
F6 / F7	Frozen requested payload, then admitted bytes to provider; response returns through the gateway.
F8 / F9	Owner revocation decision; barrier in D1 and hold/rebuild work on D2.

Use case model

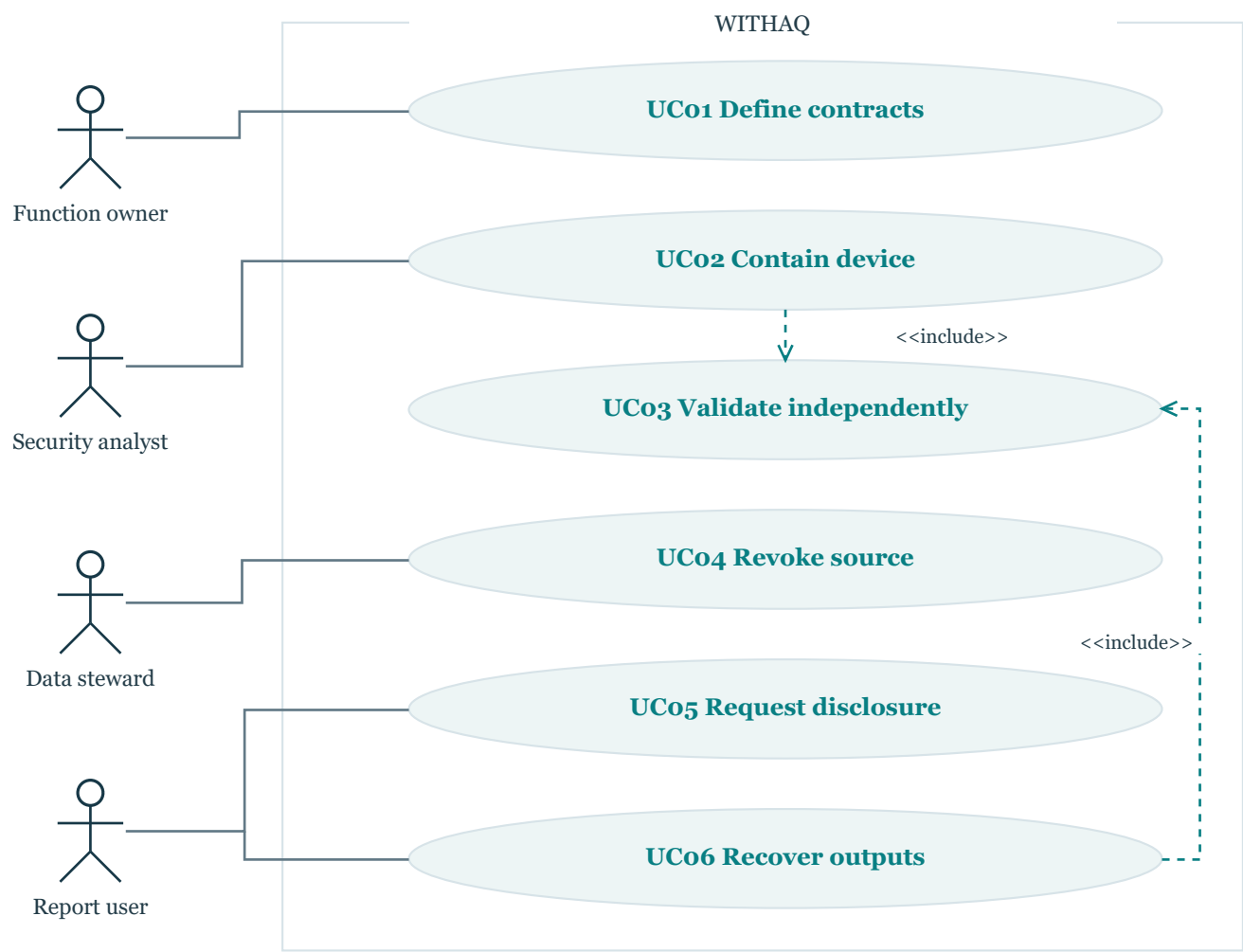


Figure 3. Use cases and system boundary. Include relationships indicate mandatory behavior; association with an actor does not grant every permission in that use case.

UCo2 containment requires an incident, approved contracts and known enforcement capabilities. It ends in confirmed execution with independent measurements, rejection or UNKNOWN. UCo4 revocation requires source-owner authority; its logical barrier completes at commit while derivative handling continues as resumable jobs.

UCo5 dispatch begins with a user request carrying a defined purpose and destination. Changes to content, roots or account after approval require reevaluation. UCo6 recovery does not undo revocation; it creates a new run and establishes permitted roots.

Actors and user stories

Permissions distinguish the function owner, data steward and policy operator. One person may hold multiple roles in the lab, but each decision records the acting role. A language model’s approval never authorizes a plan.

Story	Need and acceptance criterion
USo1	As a function owner, I need to define a critical function so its failure is not hidden by an average. A contract requires an owner, limits, measurement method and version.
USo2	As a security analyst, I need explainable containment of G. The system shows blocked paths, affected contracts and solver status before application.
USo3	As an operator, I need to see conflicts. When attack and function share an indistinguishable channel, the system reports infeasibility rather than success.
USo4	As a data steward, I need to revoke a specific batch. After commit, managed dependent reads and publication are rejected.
USo5	As a report user, I need to send an allowed summary. The user sees the denial reason or removed fields, destination and account before approval.
USo6	As a task owner, I need recovery from an alternative. The system creates a new identity and lineage without reactivating the revoked source.
USo7	As an auditor, I need to reconstruct a decision. Plan and contract versions, enforcement results and probes can be exported without sensitive payloads.
USo8	As an engineer, I need safe restarts. Duplicate events and worker recovery cannot remove restrictions or publish stale results.

Functional requirements: devices and containment

ID	Required behavior	Acceptance evidence
FRo1	Register device owner, role, boundaries and enforcement point; exclude unknown identities from automatic plans.	Versioned inventory and UNKNOWN state.
FRo2	Record observation and dependency provenance, time and approval; absence of traffic does not prove dispensability.	Hidden-dependency test.
FRo3	Create individual contracts with approved AND/OR dependencies and freshness, latency and loss bounds.	Reject an unmeasurable contract.
FRo4	Associate a trusted alert with an incident and attack paths with preconditions.	Rationale and evidence per attack edge.
FRo5	Enumerate bounded deny actions and return correct solver status and reasons.	Feasible, infeasible and timeout cases.
FRo6	Independently recheck attack reachability, every contract, enforceability and versions.	Reject an injected unsafe plan.
FRo7	Apply an authorized plan, confirm network rules and probe new and existing flows.	Ruleset snapshot, packets and measurements.
FRo8	Reevaluate recovery and expiry without automatically reopening a prohibited path.	Controller-loss test.

All these requirements are mandatory for the reference prototype. Advanced anomaly detection is deferred. The first version may use a recorded incident injection or a simple detection rule so response quality can be assessed independently of an unevaluated detector.

Functional requirements: data and integration

ID	Required behavior	Acceptance evidence
FR09	Link source batches to device and time window, while supporting institutional sources with no device.	Known and unknown impact scopes.
FR10	Seal a manifest of all run inputs and root sources before processing.	Recorded retrieval, context and tools.
FR11	Guard read, publish and dispatch against current state and versions.	Reject a result after concurrent revocation.
FR12	Commit source revocation atomically in the state store with a use barrier and durable job event.	Retry does not duplicate effects.
FR13	Separate proven derivatives from precautionary holds; never delete unknowns as proven dependents.	Comparison with lineage ground truth.
FR14	Rebuild from an approved alternative and publish a new object after validation.	New ID and permitted roots.
FR15	A single dispatch gateway decides allow, sanitize, block or require approval.	Payload, destination and account test.
FR16	Bind approval to exact-byte hash, destination, purpose, versions and expiry.	Changing one byte invalidates approval.
FR17	Show confirmed or unknown dispatch outcome without blind automatic retry.	Simulated lost external response.
FR18	Roles, audit, shared incident ID and independent layer states.	Trace USo1 through USo8.

Non-functional requirements

The following numbers are proposed laboratory acceptance targets, not results or production service commitments. Fix hardware and workload before evaluation. Retain maximum values and individual contract violations alongside percentiles.

ID	Property and target	Measurement
NFR01	Decision integrity: zero unsafe acceptances in the defined fault corpus; no extrapolation beyond it.	Independent validator and oracle.
NFR02	Responsiveness: target planning plus validation p95 below 2 s at 12 actions and 50 assets.	Separate from rule installation and probing.
NFR03	Function: one sample/s, age at most 2 s; every test alert delivered within 2 s.	Independent probes and measured clock offset.
NFR04	No revival: zero new managed read admissions or publications of a revoked root after the transaction barrier.	Concurrency and restore tests.
NFR05	Reliability: no committed revoke job lost on worker kill; resume with stable IDs.	Kill and restart at each persistence point.
NFR06	Observability: every decision has versions, reasons and correlation ID, without sensitive text by default.	Log-field audit.
NFR07	Security: least privilege, network isolation and secrets outside source control; no worker-to-provider access.	Reachability tests.
NFR08	Reproducibility: image digests, scenario seeds and retained policy versions.	Replay a reference suite.

Test the interface in Arabic and English, with explicit UNKNOWN labels and reasons. The reference journey uses three screens: incident, impact and evidence. Assess task completion in a small usability study before accepting the interaction design.

Function contracts and dependency graphs

The decision model contains attack graph A, function graph F, action set U and a mapping from each action to an actual enforcement capability. An attack edge represents a possible exploit or transition with preconditions; not every network connection is an attack edge. A function dependency describes required flows, services and limits, supported by owner evidence or tests rather than inference alone.

Lab contract	Conditions and evidence
FC1	Monitoring: one sample per second; newest-sample age at most 2 s; independent generator compares sequence and time.
FC2	Alerting: every injected alert arrives within 2 s; each loss is a distinct failure regardless of mean latency.
FC3	Noncritical archive: delay up to 30 s with retransmission; its loss may be ranked after hard constraints.

AND requires every dependency. OR requires an approved, reachable and currently measured alternative, not merely a path on a diagram. Each contract records owner, version, criticality, measurement method, window, limits, dependencies and evidence bounds. All critical contracts must pass; an aggregate availability score cannot conceal alert failure.

When a dependency is uncertain, construct plausible hypotheses H and require success across them. H does not represent every possible world; if relevant uncertainty cannot be bounded, return UNKNOWN. Refresh evidence after topology or service changes or validity-window expiry.

MFSC algorithm and decision rules

The planner searches for deny set C drawn from U . In the prototype, U contains defined binary actions such as blocking a destination or a known administrative flow. All other traffic remains subject to baseline policy; the planner never creates a broad allow rule. “Minimum” refers to the best candidate within the fixed model and actions, not an unrestricted graph-cut theorem.

FORMAL SPECIFICATION

```
C subset_of U
For every h in H:
  Reach(A_h after C, protected_targets) = false
  For every critical function f:
    Contract(f, F_h after C) = true
  Enforceable(C, capability_version) = true
Choose lexicographic minimum of:
  (action_cost, noncritical_loss, change_count, stable_id)
```

Filter feasibility first, then rank acceptable plans lexicographically by cost, noncritical loss, change count and a stable tie-breaker. Lower cost cannot compensate for a critical-contract violation. With at most 12 actions, exhaustive enumeration considers 4,096 subsets; evaluation time still depends on graphs and hypotheses, so this count does not guarantee a deadline.

BOUNDED PLANNER

```
best = None
for C in enumerate_subsets(U):
  if deadline_exceeded():
    if best is not None: return FEASIBLE(best)
    return UNKNOWN
  if all_constraints_hold(C, snapshot):
    best = lower_rank(best, C)
if best is not None: return OPTIMAL(best)
return INFEASIBLE
```

OPTIMAL means enumeration completed over the stated finite set. FEASIBLE means a plan passed constraints without an optimality proof. INFEASIBLE requires completed search with no solution. UNKNOWN covers timeout without a verified plan or insufficient evidence. Every executable plan needs independent validation, even if labeled OPTIMAL.

Worked decision example and boundaries

This is a computed illustrative example, not a WITHAQ prototype result. FC1 and FC2 require G-M and G-A. The modeled attack paths are G-P and G-X-P. Action a blocks G-P at cost 1; b blocks G-X at cost 1; c isolates G at cost 1; d blocks X-P at cost 2. All actions are deny-only; this example has no noncritical loss.

Candidate	Security and function verdict	Rank
{a}	G-X-P remains; reject.	(1,0,1)
{c}	Blocks attack but breaks both contracts; reject.	(1,0,1)
{a, b}	Blocks both paths and preserves both contracts; best.	(2,0,2)
{a, d}	Valid, higher cost.	(3,0,2)
{a, b, d}	Valid with redundant restrictions.	(4,0,3)

The Python check used during report generation enumerates 16 subsets: three are feasible and {a, b} is selected. Adding a hypothesis that G-X is critical invalidates this choice but may leave {a, d} feasible. If G-P itself is required by a critical contract and blocking is the only capability, no solution exists within the model.

What connectivity preservation cannot prove

A device may transmit a false reading over an allowed channel. MFSC does not repair firmware or establish measurement truth. This threat needs an independent source, application validation or alternative operating procedure; otherwise the system reports conflict rather than promising a safe connected device.

Enforcement, validation and recovery

The validator recomputes reachability, evaluates every contract over declared hypotheses and checks action support, enforcement identity and topology/policy versions. It shares data schemas with the planner, but uses independently designed oracle cases to reduce shared mistakes. VERIFIED applies to a specific snapshot and is not permanent execution authority.

NETWORK PLAN LIFECYCLE

```
PROPOSED -> VERIFIED -> AUTHORIZED -> APPLYING
APPLYING -> CONFIRMED | FAILED | UNKNOWN
CONFIRMED -> REEVALUATE on drift or expiry
REEVALUATE -> new verified plan (never blind reopen)
```

Before execution, a short transaction checks incident and plan versions and records an outbox intent. The adapter rechecks versions before installing rules and uses a stable operation ID. A reconciler rechecks state if it changes during application; database-to-kernel distributed atomicity is not assumed. The system remains APPLYING or UNKNOWN until enforcement and measurement evidence arrives.

Probe both established and new traffic. An ESTABLISHED accept rule must not bypass a higher-priority incident deny. The adapter must not indiscriminately flush all lab sessions; session actions are targeted and contract-checked. Initially, rules are confined to a dedicated namespace and cannot modify the host network.

Recovery is a new plan subject to the same constraints. Expiry triggers reevaluation, not rule removal. On controller loss, retain the last approved restrictions and follow the function owner's predefined failure procedure. This does not imply retaining restrictions is safe for every device; real deployment requires a specific operational assessment.

Generative AI disclosure guard

The guard asks whether these exact bytes, derived from these roots, may leave for this destination and account for this purpose now. A domain allowlist or personal-data detector alone is insufficient. Policy inspection covers the complete assembled request, including conversation history, tool outputs and contextual files.

Decision	Behavior
ALLOW	Freeze payload and check current versions and permission before dispatch.
SANITIZE	Remove or transform fields locally; create new bytes and hash and reinspect. Sanitization does not erase source lineage.
BLOCK	Reject this request with a clear reason; changed inputs or policy require a new evaluation.
REQUIRE_APPROVAL	Human review of a specific payload where policy permits; it cannot override a revoked root or hard prohibition.

Content inspection combines field rules, organizational dictionaries, fingerprints and entities. Test Arabic, dialectal and mixed text and obfuscated identifiers on a labeled synthetic corpus. Keywords are signals, not proof. Local pre-dispatch inspection avoids sending sensitive content to an external model merely to ask whether it is sensitive.

Deployment blocks direct external connections from the worker and application. This covers the integrated enterprise path; it does not establish visibility into every Claude, OpenAI or other application on user devices. An enterprise proxy or endpoint controls would require a separately defined extension.

MAHW: revocation, lineage and recovery

MAHW is a WITHAQ layer, not a file-delete button. A Root represents the authorization state of a data origin; an Artifact is an immutable document, chunk, context or result. A Run records all inputs in a Manifest and each derivative retains its transitive roots. Held roots cannot be used pending review; revoked roots never return to ACTIVE.

REVOCATION TRANSACTION

```
BEGIN
  lock roots in stable ID order
  authorize actor and check expected_version
  root.state = REVOKED; root.version += 1
  insert revocation(event_id, root_id, reason)
  insert outbox(job_id, event_id, PENDING)
COMMIT # authoritative use barrier
```

Current roots are checked at every read, publication and dispatch admission. Blocking therefore begins at commit even before descendant enumeration finishes. The worker derives D, the proven impact set, and C, the precautionary hold scope where uncertainty remains. C-D stays held for review rather than being deleted as a proven derivative.

Handling includes retrieval indexes, context stores, application memory and outputs served by the application. Revoked chunks remain unusable even while index cleanup is pending. Previously downloaded or externally stored copies require a separate process; the guard cannot recall a copy beyond its control.

Recovery uses a fresh run from an approved alternative independent of the revoked root, a new STAGED output, quality/contract/lineage checks and a short atomic promotion inside the state store. Removing a source name from a prompt or editing the old output is insufficient. The audit retains the replacement rationale without copying revoked content into general logs.

Concurrency and dispatch guarantee boundaries

Read admission and revocation use short transactions locking the same roots. The guard rejects new admissions after revocation; it cannot recall bytes already delivered or leaving the service before the barrier. Equivalent ordering applies to publication and dispatch. An MVP run is one bounded stage with sealed inputs; additional retrieval or tool results require a new stage and manifest.

A long operation captures its manifest and root versions without holding a database transaction during a model call. On return, it checks current state; a result depending on a revoked version cannot publish. Output roots are recorded before publication, with stable root-lock ordering at concurrency barriers.

EGRESS ADMISSION

```
freeze exact bytes + manifest + destination + purpose
inspect locally; obtain bound approval if required
BEGIN
  lock roots in stable ID order
  verify active versions, policy, hash and expiry
  insert egress_job(state=ADMITTED, payload_hash=...)
COMMIT
send those bytes once through the gateway
record SENT | FAILED | OUTCOME_UNKNOWN
```

The ordering point is transactional ADMITTED. Revocation before that point rejects the request. Revocation after it may occur after transmission has begun and cannot guarantee recall. The worker attempts to cancel work not yet started, but the report does not promise to cancel in-flight bytes. Audit records revocation, admission and send-attempt times.

If the response is lost after transmission, record OUTCOME_UNKNOWN. Automatic retry is permitted only when the adapter provides a documented, queryable mechanism that prevents duplicate effects. Exactly-once delivery to a provider outside the transaction is not a general guarantee. Changed approval, destination or bytes requires a new request.

System class diagram

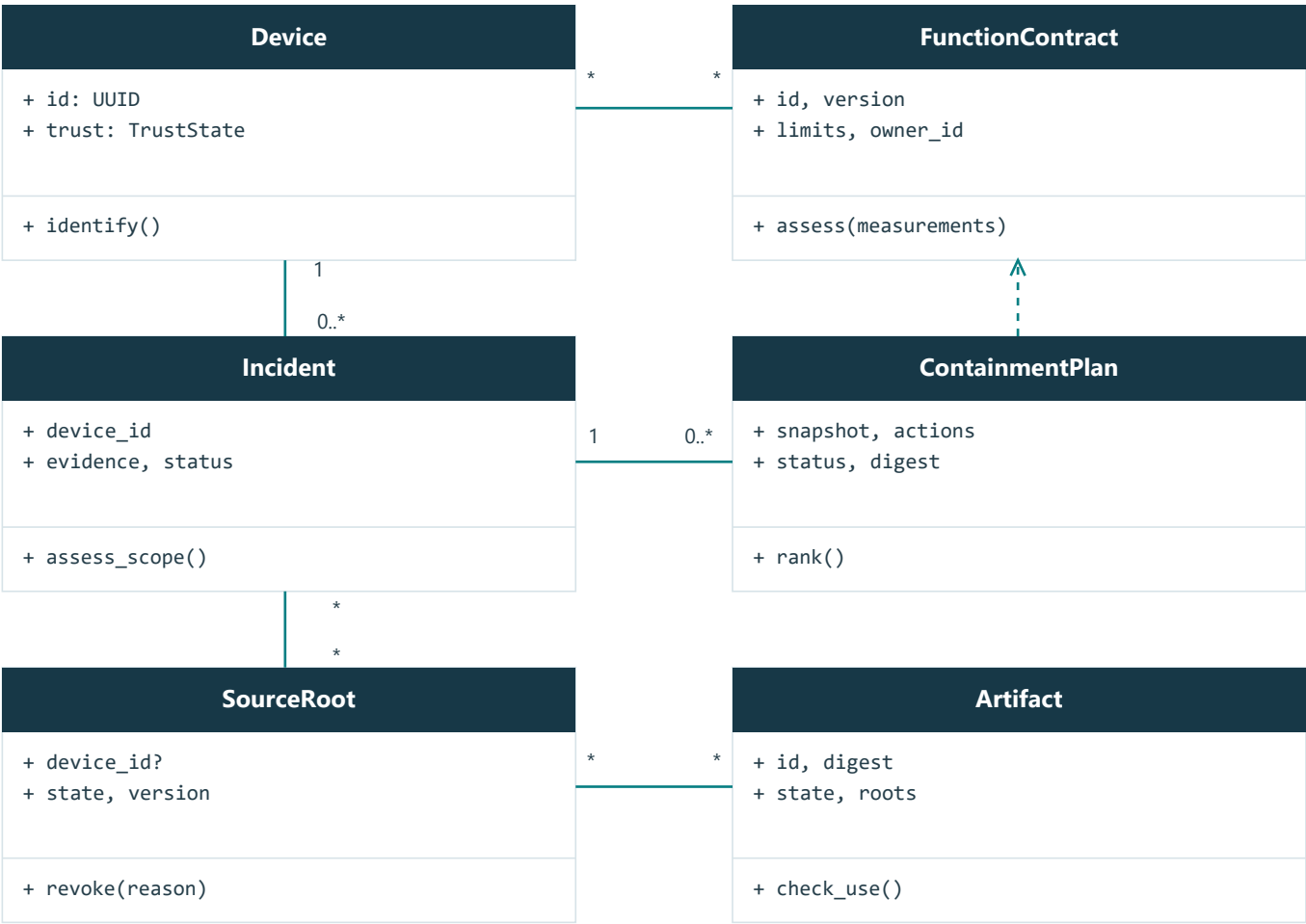


Figure 4. Domain classes and relationships. Code identifiers are stable across both editions, with localized explanation. Numbers denote multiplicity; the dashed line denotes the plan dependency on the function contract.

Device is an asset with a stable identity; FunctionContract is an immutable versioned specification; Incident records a threat assessment; ContainmentPlan binds actions to a snapshot; ValidationReport contains independent evidence. On the data side, SourceRoot controls use authorization and Artifact identifies immutable content. Planning, validation and guard services depend on these models without inheriting one another’s authority.

Shown methods specify proposed domain interfaces, not evidence of a completed repository. Objects do not grant themselves content access; services use guarded API paths. The following ER diagrams map relationships into tables and keys.

Containment sequence diagram

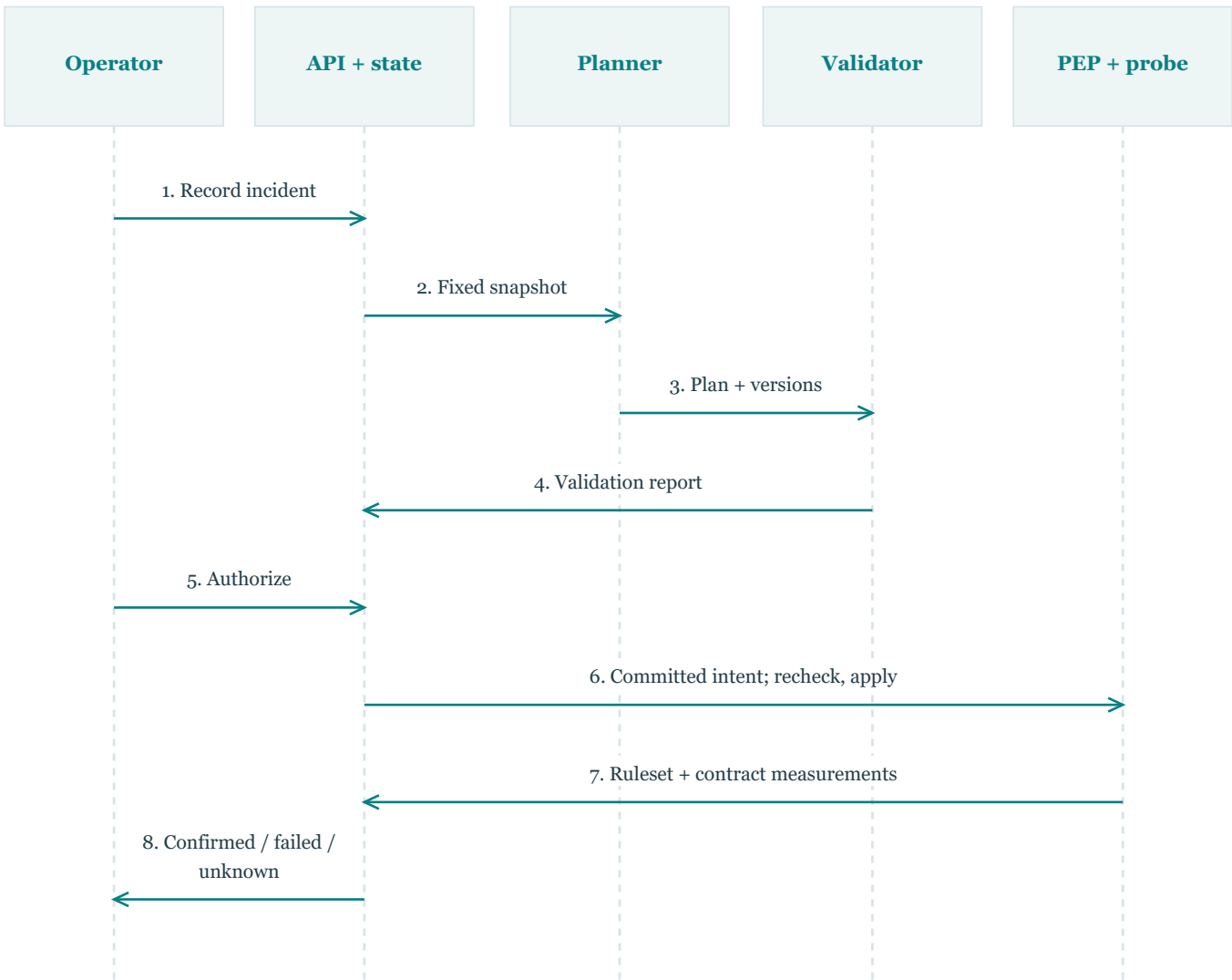


Figure 5. Containment sequence: pre-apply checks followed by ruleset confirmation and independent measurement. API success alone cannot establish functional continuity.

An alternative path rejects version or constraint failures and reports UNKNOWN when application confirmation is missing. Reconciliation distinguishes persisted intent from observed rules. Automatic execution cannot let an operator bypass a critical contract; changing it is a separate owner decision with a new version and evidence.

Revocation and recovery sequence diagram

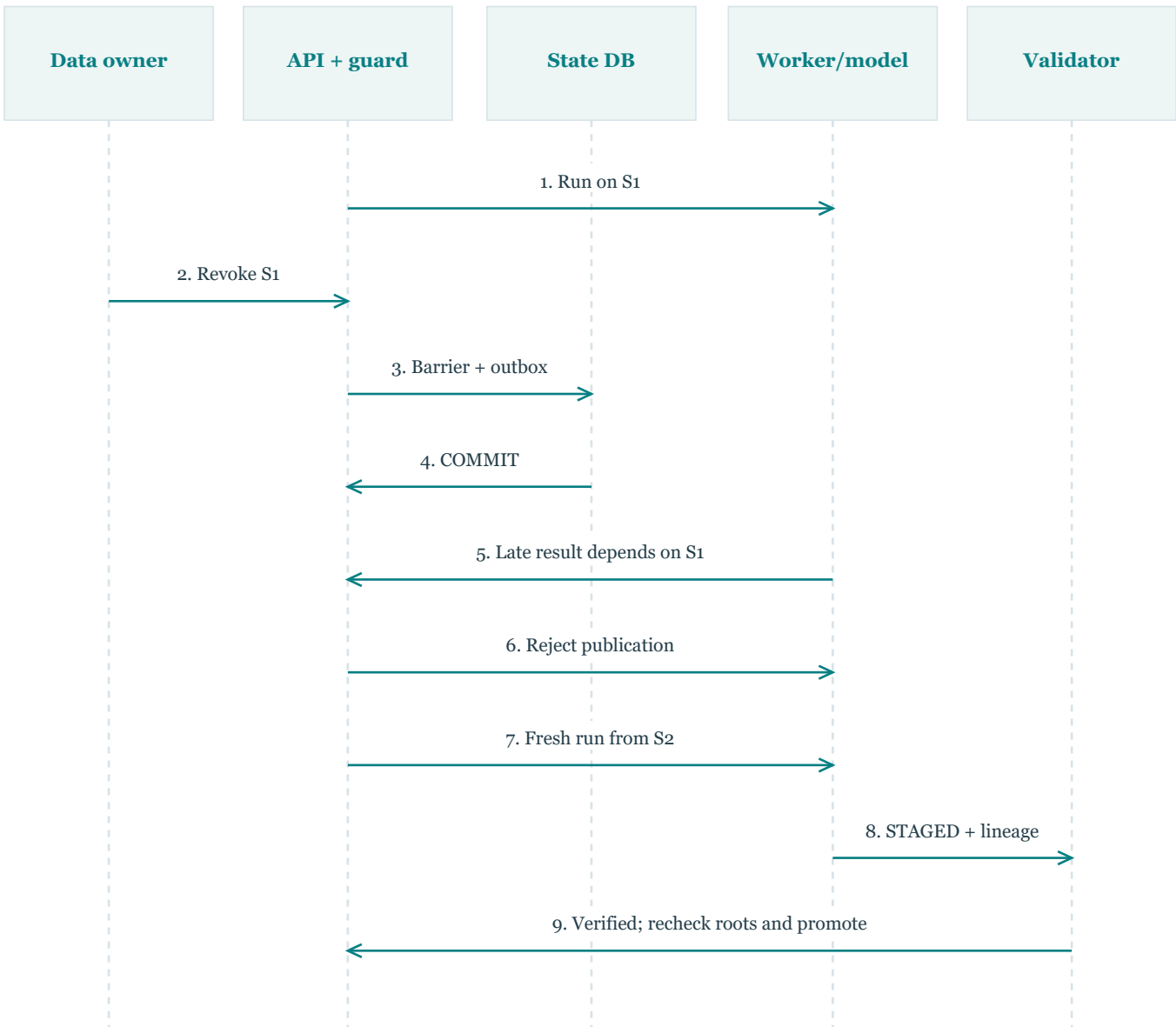


Figure 6. Run/revocation race: the state barrier precedes descendant cleanup and blocks a late result. Recovery creates a new output.

No-revival depends on authoritative root guards, not on fast worker completion. A worker crash after COMMIT resumes from the outbox, while a model returning after revocation fails publication checks. Recovery from S2 neither removes S1’s revocation history nor erases investigative evidence.

Entity-relationship model: network

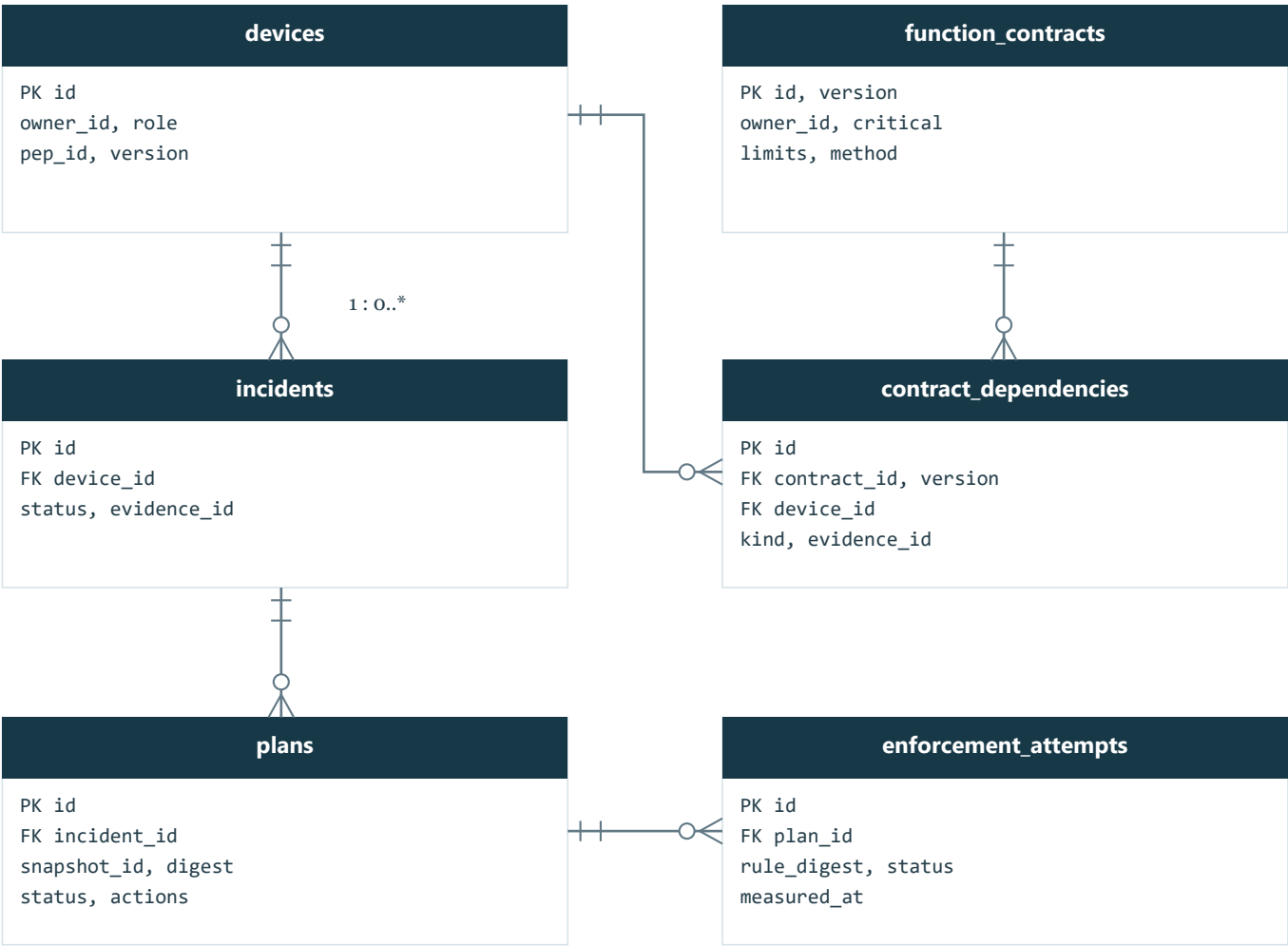


Figure 7. ER diagram for devices and response. PK denotes primary key and FK foreign key; crow’s feet denote many and circles denote optionality.

Devices have incidents and contract associations. contract_dependencies explicitly links contracts to dependent resources with AND/OR semantics and evidence. Each plan stores a snapshot ID, digest, versions, validator reference and solver status. enforcement_attempts preserves attempts instead of overwriting history; a plan may have several attempts, but only confirmation for its authorized version determines current state.

Entity-relationship model: data

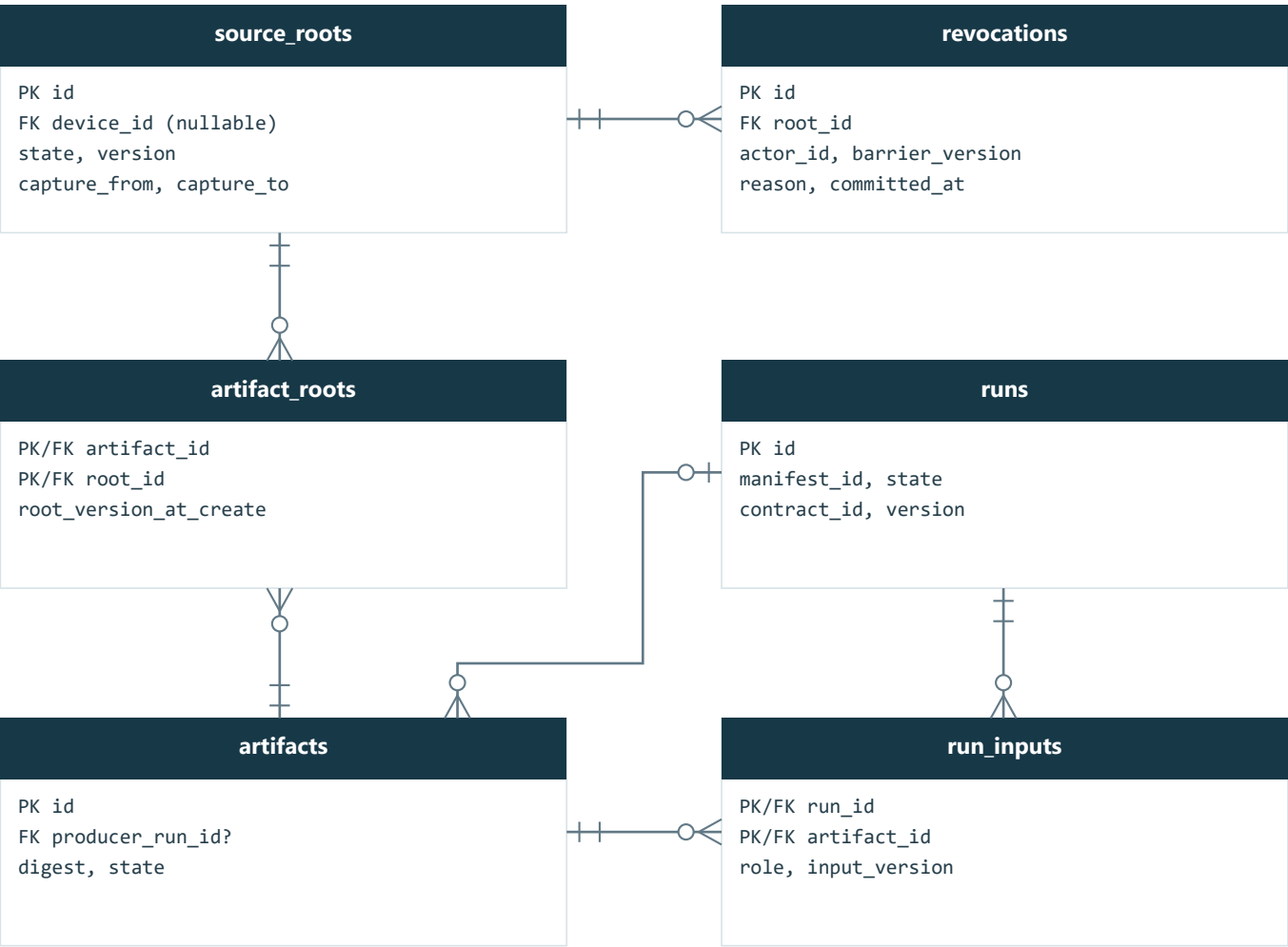


Figure 8. ER for sources, execution and lineage. artifact_roots is a many-to-many bridge; root lineage is not inferred from text similarity.

source_roots includes an optional device ID, capture window, classification, state and version. Institutional sources may have no device. artifacts have immutable content and digests; run_inputs records every consumed item. artifact_roots stores the root closure at creation, while the guard determines whether those roots are permitted now.

Unique constraints cover artifact/root pairs and event IDs. Lineage insertion rejects cycles and preserves creation ordering. Revocation must not delete the authoritative reference record and thereby make a derivative appear rootless and permitted.

Entity-relationship model: dispatch and evidence

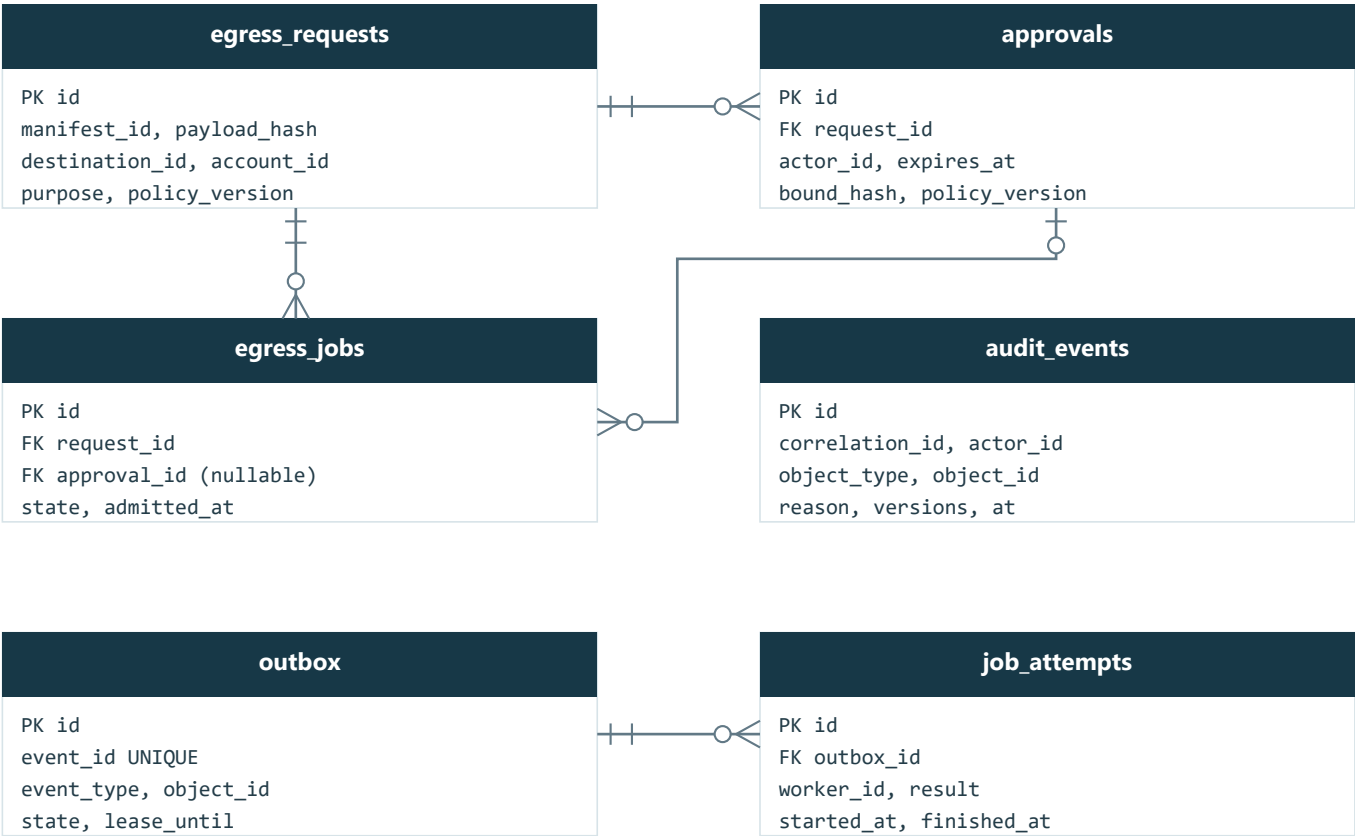


Figure 9. ER for disclosure and durable jobs. Approval is optional because policy may permit some requests without review; current guards remain mandatory.

Constraint	Implementation rule
CK-01	Approval valid for the same payload_hash, destination_id, account_id, purpose and policy_version.
CK-02	Each egress_job references a frozen request and manifest; bytes cannot change after ADMITTED.
CK-03	Unique outbox.event_id, timed worker lease and attempt counter; handlers are idempotent.
CK-04	Application append-only audit with restricted writes; protected export is a later defense against DB administrators.

API and service contracts

All routes sit under /v1 and require an authorized service or user identity. Mutating commands carry expected_version and Idempotency-Key; version conflicts fail instead of silently overwriting state. A 202 response means accepted asynchronous work, not confirmed enforcement.

Route	Request and result
POST /contracts	Function, limits and owner evidence; returns contract_id and version.
POST /incidents	device_id, evidence and time window; returns incident_id and suspect scope.
POST /plans	incident_id and snapshot_id; returns job_id, then solver status and evidence.
POST /plans/{id}/apply	Authorized plan version; returns attempt_id for status polling.
POST /roots/{id}/revoke	Reason, scope and authority; returns barrier_version and job_id.
POST /runs	Task contract and complete inputs; returns run_id and manifest_id.
GET /artifacts/{id}	Check current roots before returning any bytes; reject unknown or revoked roots.
POST /egress/requests	Payload, purpose, destination and account; returns decision, hash and approval request if needed.
POST /egress/{id}/dispatch	Fresh checks and admission; returns job_id and separate send state.

Domain errors include CONTRACT_CONFLICT, STALE_SNAPSHOT, ROOT_REVOKED and OUTCOME_UNKNOWN with a reason and evidence ID. Error messages contain no sensitive source text. Status pages use polling or SSE, without direct database access from the browser.

Zero-to-start: host and tools

Commands use Bash in an Ubuntu development VM. Code may be edited on Windows; use WSL2 or a VM for application development, while actual network enforcement runs in a separate lab VM. Do not mix PowerShell paths with Bash commands. Initial resource assumption: four CPU cores, 16 GB RAM and 40 GB storage, to be revised by measurement.

BASH | HOST PREPARATION

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git curl jq
python3 --version
git --version
node --version
npm --version
docker --version
docker compose version
```

Install Docker Engine and the Compose plugin using Docker's Ubuntu instructions, and install a Node version supported by the frontend tooling. The reviewed Vite guide specifies Node 20.19+ or 22.12+; select a supported release meeting that requirement and record it. Use Python 3.12 as a proposed baseline and test the dependency lock against it. The commands above expose missing tools; they do not install Docker or Node automatically. (Docker, n.d.; Vite, n.d.).

Gate 0: environment

Proceed only when version checks pass, Docker works and a Python environment can be created. Record OS and tool versions. These deployment steps were not executed on a target system during report preparation.

Repository creation and dependency locking

BASH | NEW REPOSITORY

```
mkdir withaq
cd withaq
git init
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip pip-tools
mkdir -p src/withaq apps tests fixtures policies docs
touch src/withaq/__init__.py
```

Create the following configuration files first. `.gitignore` excludes `.env`, `.venv`, `node_modules`, `var`, `dist`, `__pycache__` and local test results, while retaining lock files, migrations and synthetic fixtures. Do not commit credentials or real data. `venv` isolates project packages from the host Python environment. (Python Software Foundation, n.d.).

requirements.in

```
fastapi
uvicorn[standard]
pydantic-settings
SQLAlchemy
psycopg[binary]
alembic
networkx
httpx
PyYAML
```

requirements-dev.in

```
-c requirements.lock.txt
pytest
pytest-asyncio
ruff
```

BASH | RESOLVE, THEN REUSE LOCKS

```
pip-compile --generate-hashes requirements.in \
  -o requirements.lock.txt
pip-compile --generate-hashes requirements-dev.in \
  -o requirements-dev.lock.txt
pip install --require-hashes -r requirements.lock.txt
pip install --require-hashes -r requirements-dev.lock.txt
```

Test, commit and reuse the locks. A tested lock is produced during implementation; it is not supplied here.

First runnable service

pyproject.toml | MINIMUM BOOTSTRAP

```
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

[project]
name = "withaq"
version = "0.1.0"
requires-python = ">=3.12"
dynamic = ["dependencies"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.setuptools.dynamic]
dependencies = {file = ["requirements.in"]}

[tool.pytest.ini_options]
testpaths = ["tests"]
```

src/withaq/main.py | BOOTSTRAP ONLY

```
from fastapi import FastAPI

app = FastAPI(title="WITHAQ", version="0.1.0")

@app.get("/health/live")
def live():
    return {"status": "ok", "mode": "bootstrap"}
```

BASH | TERMINAL A

```
pip install --no-deps -e .
uvicorn withaq.main:app --host 127.0.0.1 --port 8000
```

BASH | TERMINAL B

```
curl -fsS http://127.0.0.1:8000/health/live
# Expected: {"status":"ok","mode":"bootstrap"}
```

This starts a health endpoint only; it does not implement containment, MAHW or business authorization. Add lifecycle handling, inventoried routes and /health/ready later. FastAPI describes HTTP interfaces and generates /docs; opening the documentation does not establish security completeness. (FastAPI, n.d.).

Configuration and local database

.env.example | DEVELOPMENT DEFAULTS

```
WITHAQ_MODE=mock
POSTGRES_DB=withaq
POSTGRES_USER=withaq
POSTGRES_PASSWORD=
DATABASE_URL=
WITHAQ_ADMIN_TOKEN=
WITHAQ_INTERNAL_TOKEN=
WITHAQ_API_URL=http://127.0.0.1:8000
WITHAQ_VALIDATOR_URL=http://127.0.0.1:8002
WITHAQ_GATEWAY_URL=http://127.0.0.1:8003
WITHAQ_STORAGE_ROOT=./var/objects
WITHAQ_PROVIDER=mock
WITHAQ_POLICY_PATH=./policies/disclosure.yml
```

Copy the example to `.env` and run the following once in a new repository. It generates local secrets without printing them. Do not rerun over an initialized `.env`: changing its password does not change a database user already stored in a volume. `config.py` loads settings with `pydantic-settings` and rejects empty credentials in full mode.

BASH | INITIAL LOCAL SECRETS

```
cp .env.example .env
python - <<'PY'
from pathlib import Path
from secrets import token_hex
p = Path(".env")
s = p.read_text()
pwd = token_hex(24)
values = {
    "POSTGRES_PASSWORD": pwd,
    "WITHAQ_ADMIN_TOKEN": token_hex(32),
    "WITHAQ_INTERNAL_TOKEN": token_hex(32),
    "DATABASE_URL": ("postgresql+psycopg://withaq:"
                    + pwd + "@127.0.0.1:5432/withaq"),
}
for key, value in values.items():
    s = s.replace(key + "\n", key + "=" + value + "\n")
p.write_text(s)
p.chmod(0o600)
PY
```

Static tokens are a local loopback development mechanism. Lab deployment uses distinct service identities and scoped permissions; broader deployment requires institutional identity and TLS. The worker receives no `DATABASE_URL` or PostgreSQL password, only a service credential limited to job claims and results.

Compose, migrations and readiness

compose.dev.yml | DATABASE ONLY

```
name: withaq-dev
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${POSTGRES_DB:?set POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER:?set POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set password}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL",
        "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
      interval: 5s
      timeout: 3s
      retries: 20
volumes:
  db_data:
```

BASH | DATABASE AND MIGRATIONS

```
docker compose -f compose.dev.yml up -d --wait db
docker compose -f compose.dev.yml ps
alembic init migrations
# Implement env.py and reviewed migration revisions first.
alembic upgrade head
alembic current
```

Run alembic init once only if migrations does not exist. Update env.py to load the URL from config.py and import ORM metadata, then implement and review the three listed revisions and constraints. Empty migrations do not establish readiness. depends_on alone is insufficient for service readiness; lab deployment uses healthchecks and service_healthy dependencies. (Alembic, n.d.; Docker, n.d.).

/health/live checks the process, while /health/ready checks database access, migration head and loaded policy, returning 503 when incomplete. postgres:17 is an illustrative baseline, not an immutable pin. Before lab handover, pin the image digest and test restoration against it.

Implementing the authoritative state model

Migration	Tables and required constraints
0001	devices, function_contracts, contract_dependencies, incidents, snapshots, plans, validation_reports, enforcement_attempts; UUID IDs, nonnegative versions and FKs.
0002	source_roots, source_incident_links, artifacts, lineage_edges, artifact_roots, runs, manifests, run_inputs, revocations; cycle rejection and unique lineage pairs.
0003	egress_requests, approvals, egress_jobs, outbox, job_attempts, audit_events, idempotency_keys; unique event_id and approval bound to the same request.

The preceding diagrams show core entities; this list supplies the link and snapshot tables needed for implementation. manifests stores a sealed input list and hash; snapshots stores graph representations, enforcement capabilities and contract versions. source_incident_links models incident-to-batch associations with time windows and attribution evidence.

The guard locks roots in stable UUID order for read admission, revocation, publication and dispatch. Version conflicts trigger bounded reevaluation; deadlocks and database failures never become default permission. PostgreSQL provides locking primitives, but the application protocol still needs independent race tests. (PostgreSQL Global Development Group, n.d.).

Gate 1: state and identity

A fresh database reaches head; a user lacking a role gets 403; a stale version gets 409; reuse of an Idempotency-Key with different bytes is rejected. Do not build destructive cleanup or dispatch paths before this gate.

Building planner, validator and worker

Start fixtures/network.json with the earlier worked example, then implement planner.py as a pure Snapshot-to-PlanResult function. Keep database reads and wall-clock access outside feasibility checks; inject the deadline and clock into the search loop. best is None distinguishes no solution from a valid empty deny set. stable_id ensures deterministic ties.

INTERFACE CONTRACTS | PROPOSED

```
plan(snapshot, deadline) -> PlanResult
validate(snapshot, candidate) -> ValidationReport
claim_job(worker_id) -> JobLease | None
heartbeat(job_id, attempt_id, lease_token) -> Lease
complete(job_id, attempt_id, lease_token, result) -> Ack
reconcile(plan_id, observed_rules) -> EnforcementState
```

Implement validator/checks.py against independent oracle cases without importing planner feasibility functions. Recheck contracts, dependency hypotheses, reachability, actions and versions. Reports carry snapshot and plan hashes so a valid report cannot authorize a different plan. A separate process alone does not establish logical independence.

The API leases outbox jobs with attempt_id and a fencing token. Workers heartbeat and stale completions are rejected. Handlers are idempotent and resume from durable steps. Model calls and long validation never hold an open database lock.

BASH | AFTER MODULES ARE IMPLEMENTED

```
python -m withaq.validator.main
# In a separate terminal, same repository and virtualenv:
python -m withaq.worker.main
# Verification:
pytest tests/unit/test_planner.py -q
pytest tests/unit/test_validator.py -q
pytest tests/integration/test_job_restart.py -q
```

Gate 2: decisions

The example selects {a, b}; alert-breaking plans fail; timeouts are not INFEASIBLE; job retries do not create uncontrolled duplicate applications. Mock-adaptor outcomes remain explicitly simulated.

Building MAHW and the disclosure gateway

Implement lineage.py before source upload screens: every Artifact has complete roots or UNKNOWN status. A stage manifest seals every input, including permitted system context, conversation history and tool results. Input changes require a new stage. Sanitizing text never grants permission to discard source lineage.

Implementation order	Completion condition
1. guards.py	Every read and publication passes current-root guards.
2. revocation.py	Revoke, version and outbox commit together; state cannot revert to ACTIVE.
3. recovery.py	Independent S2 alternative; new STAGED output; validate then promote.
4. disclosure.py	Complete local byte/destination/account/purpose inspection; sanitized bytes are reinspected.
5. admission.py	Hash, approval, versions and locked roots; persist ADMITTED before networking.
6. gateway/main.py	Send identical bytes; record unknown outcome on lost response without blind retry.

BASH | AFTER IMPLEMENTATION

```
python -m withaq.gateway.main
# In another terminal:
pytest tests/unit/test_lineage.py -q
pytest tests/integration/test_revoke_race.py -q
pytest tests/integration/test_egress_race.py -q
```

Gate 3 requires S1 revocation to block T1, T2 and late model output while proven-independent T3 continues and T2b is rebuilt from S2. Gate 4 adds dispatch blocking, post-approval byte mutation and lost-response handling. All examples use synthetic content and require no paid provider key.

Building and connecting the console

BASH | CREATE FRONTEND ONCE

```
npm create vite@latest apps/console -- --template react-ts
cd apps/console
npm install
npm install -D @playwright/test
npx playwright install chromium
# Commit the generated package-lock.json after checks.
npm run dev -- --host 127.0.0.1 --port 5173
```

Use the create command once in a new directory, never over an existing application. Thereafter npm ci reuses the lock. Add inventoried files and remove unused starter assets. Never place administrator tokens or provider keys in VITE_* variables, which are exposed to the browser. (Vite, n.d.).

apps/console/vite.config.ts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    host: "127.0.0.1",
    port: 5173,
    proxy: {
      "/v1": "http://127.0.0.1:8000",
      "/health": "http://127.0.0.1:8000",
    },
  },
});
```

Incident.tsx separates plan state, enforcement confirmation and contracts. DataImpact.tsx distinguishes proven derivatives from precautionary holds. Disclosure.tsx shows final bytes, destination, account and purpose before approval. A development user enters a token for the current session, held only in memory and reentered after refresh; institutional identity replaces this mechanism in deployment.

BASH | CONSOLE CHECKS

```
cd apps/console
npm run build
npx playwright test
```

Gate 5: users can recognize infeasibility and contract failure without relying on color. Server authorization still rejects a request from a modified browser. The console displays API judgments and evidence rather than reimplementing MFSC or source authorization.

Local startup in the correct order

Use this page after Gates 1-5 and the entry points are implemented. It distinguishes a working bootstrap service from a complete WITHAQ run. Each Python terminal starts at the withaq root and activates .venv; processes load their own configuration without logging secrets.

Order	Command from project root	Check
1	<code>docker compose -f compose.dev.yml up -d --wait db</code>	Database healthy.
2	<code>alembic upgrade head</code>	Migrations at head.
3	<code>uvicorn withaq.main:app --host 127.0.0.1 --port 8000</code>	live and ready return 200.
4	<code>python -m withaq.validator.main</code>	Internal check on 8002.
5	<code>python -m withaq.gateway.main</code>	Mock provider on 8003.
6	<code>python -m withaq.worker.main</code>	Fresh heartbeat and valid claim.
7	<code>python -m withaq.cli seed --fixture fixtures</code>	Synthetic records only.
8	<code>npm --prefix apps/console run dev -- --host 127.0.0.1</code>	Console on 5173.

BASH | OPERATIONAL CHECKS

```
curl -fsS http://127.0.0.1:8000/health/ready
python -m withaq.cli check
python -m withaq.cli demo --scenario separable
python -m withaq.cli demo --scenario revoke-race
```

cli.py must explicitly implement these commands: seed performs deterministic idempotent ingestion through the API; check inspects dependencies and jobs; demo executes a recorded request sequence. These names are proposed implementation contracts, not tools already installed on your machine. Complete a missing file or route’s gate before attempting startup.

Real network enforcement laboratory

Mock mode demonstrates decision logic and integration only. Traffic-denial evidence requires a separate VM and a PEP that covered flows cannot bypass. `lab/control.py` creates resources prefixed with `withaq-lab` and records their identities. It must never flush host rules or change the host default route. `down` removes only verified, recorded lab-owned resources.

lab/topology.yml | PROPOSED ADDRESS PLAN

```
lab_id: withaq-lab
subnet: 10.77.0.0/24
nodes:
  gateway: {ip: 10.77.0.10}
  monitor: {ip: 10.77.0.20, port: 9001}
  alert: {ip: 10.77.0.21, port: 9002}
  archive: {ip: 10.77.0.22, port: 9003}
  protected: {ip: 10.77.0.30, port: 9443}
contracts:
  monitor_max_age_ms: 2000
  alert_max_latency_ms: 2000
  archive_max_delay_ms: 30000
```

Addresses are a logical plan, not instructions to put all nodes on a bridge that bypasses enforcement. `control.py` creates separate links whose traffic crosses the enforcement namespace; `probes.py` verifies this before experimentation. Measure a baseline, inject a synthetic incident, block G-P while preserving G-M/G-A, then test an infeasible shared-channel case.

BASH | DEDICATED LAB VM, AFTER IMPLEMENTATION

```
sudo apt install nftables iproute2
sudo .venv/bin/python lab/control.py up \
  --config lab/topology.yml
.venv/bin/python lab/control.py status
.venv/bin/python lab/probes.py --suite baseline
.venv/bin/python lab/probes.py --suite containment
pytest tests/lab/test_enforcement.py -q
```

The API never runs as root. The PEP accepts predefined authenticated actions, not user-supplied shell text. Compare read-back rules with the plan digest and probe both existing and new sessions. Contract observation remains separate from planning. Namespace privileges and kernel interfaces must be implemented and reviewed before using the proposed commands.

Lab deployment, networks and credentials

Network	Proposed members	Access rule
app	console / API	Expose the console only; application requests go through API.
control	API / worker / validator / gateway / PEP	Internal service identities and timeouts; no direct worker egress.
data	API / PostgreSQL	Internal; no published PostgreSQL port in lab deployment.
egress	gateway / approved endpoint	Only the gateway reaches policy-approved external endpoints.
device-lab	PEP / synthetic devices / probes	Lab data paths only; separate management connectivity.

compose.lab.yml is a later implementation artifact containing db, api, worker, validator, gateway, pep and console. internal: true provides basic isolation but does not replace destination policy or bypass testing. Remove external DB ports and use distinct DB privileges and TLS when leaving the local host. The PEP gets narrowly scoped lab-VM privileges and no Docker socket.

BASH | ONLY AFTER LAB FILES AND IMAGES EXIST

```
docker compose -f compose.lab.yml config --quiet
docker compose -f compose.lab.yml build
docker compose -f compose.lab.yml up -d --wait
docker compose -f compose.lab.yml ps
pytest tests/security/test_bypass.py -q
```

Deployment acceptance

A successful Compose startup is not containment evidence. Pass PEP-path, established-session denial, functional-contract, worker-egress and revocation-guard tests. If a bypass remains, do not claim coverage of all data.

Complete demonstration and expected evidence

Stage	User action	Expected evidence
A	Load the G, S1, S2 and S3 fixture.	IDs, versions, contracts and measured baseline.
B	Inject a synthetic G incident.	Specific plan and hash-bound validator report.
C	Authorize the plan and wait for measurements.	Denial to P with FC1/FC2 passing, or explicit infeasibility.
D	Request disclosure containing prohibited fields.	BLOCK or reinspected sanitization; no hidden send.
E	Revoke S1 during a delayed model call.	Revoke barrier then publication denial; proven T3 continues.
F	Rebuild T2 from S2.	New T2b; quality, lineage and promotion checks.
G	Stop and restart the worker.	Jobs resume without source revival or unknown-send duplication.

Each result bundle includes run_id, commit, policy versions, measurements and per-test verdicts. A console video may explain the story but does not prove packet behavior or prevent revoked-source reuse. Mock results are explicitly labeled simulation.

Acceptance tests and traceability

Test	Scenario and expected result	Trace
ATo1	Separable incident; denial confirmed and every critical contract passes.	O2 / USo2 / FRo5-07
ATo2	Required attack channel; INFEASIBLE after complete search, no fabricated success.	O2 / USo3 / FRo5
ATo3	Unknown dependency; hold or UNKNOWN until sufficient evidence.	O1 / USo1 / FRo2-03
ATo4	Tampered plan or stale snapshot; validator/guard rejects.	O2 / FRo6 / NFRo1
ATo5	Established session to a denied target; restrictions stop it too.	O2 / FRo7 / NFRo3
ATo6	Revocation during model call; late result cannot publish.	O4 / USo4 / FR11-12
ATo7	Revoke S1; block T1/T2, keep proven T3, do not delete C-D.	O4 / FR13 / NFRo4
ATo8	Recover T2; new identity with S2 roots and passing quality checks.	O5 / USo6 / FR14
ATo9	Change account or byte after approval; reject dispatch.	O3 / USo5 / FR16
ATo10	Revocation before ADMITTED blocks; afterward cancellation limits are visible.	O3-4 / FR17
ATo11	Kill worker or duplicate event; no loss/revival; DB restore replays revocations first.	USo8 / NFRo5
ATo12	Lose controller; no automatic unblocking or undocumented success state.	USo8 / FRo8

Add interface integration tests for roles, logging, direct gateway bypass and reason display. Every test records trial ID, versions, inputs, raw results and expected/actual verdicts. This is an acceptance plan; no test is marked passed until the system is executed and evidence collected.

Operations, observability and failure recovery

The incident screen presents separate device-trust, plan, enforcement-confirmation, critical-contract and data/disclosure states. Metrics include decision/install times, sample ages, missing messages, queue depth and version-related rejections. Records carry incident_id, plan_id or run_id, policy_version and reason.

Failure	Proposed behavior
State-store failure	Stop sensitive reads, publication and new dispatch; cached state cannot assert fresh permission. Installed network rules remain.
Worker failure	Lease expires and another worker resumes from the last durable step using stable IDs.
Network-adapter loss	UNKNOWN or explicit stale state; read back rules on recovery before admitting a new plan.
Delayed index cleanup	Authoritative guards reject revoked results even while index entries remain.
Backup restore	Start isolated; load a newer trusted revocation ledger; rebuild barriers and test before reopening.

Test revocation-ledger export to a separate store before relying on old-backup recovery. If a newer trusted ledger is unavailable, restored content cannot be proven permitted and remains held. Production operations later require retention policy, secret rotation, access review and recurring restore tests.

Stop, resume and backup procedures

For graceful shutdown, stop new run and dispatch admissions, let workers finish or persist their stage, then stop application services and the database. Do not use `down -v` for routine shutdown because it deletes volumes. PEP restrictions remain during controller loss until a reviewed action; administrative shutdown must not automatically remove attack restrictions.

BASH | DEVELOPMENT DATABASE BACKUP

```
mkdir -p var/backups
docker compose -f compose.dev.yml exec -T db \
  pg_dump -U withaq -d withaq -Fc \
  > var/backups/withaq.dump
# Stop app terminals gracefully, then:
docker compose -f compose.dev.yml stop db
```

The dump covers database state only. Include object storage, a newer trusted revocation ledger and policy versions in a consistent restore set, with encrypted backups and restricted access. Restore into a new isolated environment, apply compatible migrations and replay revocations before enabling reads or dispatch. `test_restore.py` verifies that an old backup cannot revive a revoked root.

The runbook records backup time, schema/service versions and ledger position. If ledger freshness cannot be established, restored data remains held. MAHW does not automatically delete forensic records; evidence retention permission is separate from permission to use content in a report or model.

Troubleshooting common failures

Symptom	Check and action
ModuleNotFoundError	Activate .venv and run pip install --no-deps -e . at the root; avoid arbitrary PYTHONPATH changes.
DB connection refused	Check Compose status, health and address; use 127.0.0.1 locally and db inside containers.
DB password rejected	Check .env against the existing volume; editing env does not recreate a user.
ready returns 503	Check migration head, database access and policy loading; live alone is insufficient.
Console loads, requests fail	Inspect proxy, route and session token; 403 is permission and 409 a version conflict.
Plan UNKNOWN	Inspect deadlines and missing evidence; do not relabel as safe or impossible.
Rule applied, traffic remains	Check PEP path, established-rule order and IPv4/IPv6 traffic; adapter acknowledgment is not proof.
Job stuck	Inspect lease, heartbeat and attempts; resume with the same identity and version guards.
Revoked data still displayed	Distinguish previously downloaded bytes from new reads; inspect guards, UI cache clearing and roots.
Unknown send outcome	Do not retry blindly; check provider adapter and request identity before a documented operator decision.

Complete MVP file-tree contract

The following inventory specifies 161 authored or generated files for the proposed MVP. It is a target structure contract, not an inventory of an already implemented application. Every listed file has a name and responsibility; required files are not hidden behind ellipses. Add new files only with an inventory update.

withaq/ | TOP-LEVEL MAP

```
withaq/  
  .github/workflows/  
  apps/console/  
  src/withaq/  
    http/routes/  
    domain/  
    schemas/  
    services/  
    repositories/  
    adapters/  
    worker/  
    validator/  
    gateway/  
    pep/  
  migrations/versions/  
  lab/nftables/  
  fixtures/  
  policies/  
  tests/unit/  
  tests/integration/  
  tests/security/  
  tests/lab/  
  experiments/  
  docs/adr/
```

Directories `.venv`, `node_modules`, `dist`, `__pycache__` and `var` contain generated or local data, so thousands of dependency-internal files are not enumerated. `var` holds objects, backups, results and local logs. Generated lock files are committed; credentials, real content and sensitive results are not.

File inventory: Root and configuration

Base path: withaq/ (root)

File	Responsibility
README.md	Project entry point, startup commands and release boundaries.
.gitignore	Exclude secrets, environments, local data and builds.
.env.example	Configuration names and nonsecret development defaults.
pyproject.toml	Python package, src layout and test settings.
requirements.in	Direct runtime dependencies before resolution.
requirements-dev.in	Testing and dependency-resolution inputs.
requirements.lock.txt	Generated hashed runtime dependency lock.
requirements-dev.lock.txt	Generated development lock compatible with runtime lock.
alembic.ini	Database migration paths without credentials.
compose.dev.yml	Local PostgreSQL; application runs in Python environment.
compose.lab.yml	Isolated lab deployment after service images exist.
Dockerfile.api	Build an unprivileged API image.
Dockerfile.worker	Build a worker image without network administration rights.
Dockerfile.gateway	Dispatch-gateway image; inject secrets at runtime.
Dockerfile.validator	Independent validator image without a public listener.
Dockerfile.pep	Enforcement-agent image restricted to the lab.
.github/workflows/ci.yml	Build and test migrations, guards and console.

File inventory: Application core and routes

Base path: src/withaq/

File	Responsibility
__init__.py	Package identity and application version.
main.py	Create FastAPI, register routes and lifecycle.
config.py	Load typed configuration and validate execution mode.
db.py	Database engine, sessions and transaction boundaries.
security.py	Identity, roles and service-token checks.
logging.py	JSON logging with content and secret redaction.
cli.py	Seed, demo and check commands through the API.
http/__init__.py	HTTP package marker.
http/dependencies.py	Inject identity, sessions and correlation IDs.
http/errors.py	Map domain errors to stable HTTP responses.
http/routes/__init__.py	Collect authorized routers.
http/routes/health.py	Separate process liveness and dependency readiness.
http/routes/devices.py	Device inventory and versioned updates.
http/routes/contracts.py	Define and approve contract revisions.
http/routes/incidents.py	Record incident evidence and impact window.
http/routes/plans.py	Request, validate and authorize plans.
http/routes/sources.py	Register roots, holds and revocation.
http/routes/runs.py	Admit runs with sealed inputs.
http/routes/artifacts.py	Guard artifact reads, publication and recovery.
http/routes/egress.py	Inspect, approve and admit disclosure requests.
http/routes/evidence.py	Role-scoped evidence viewing and export.
http/routes/internal.py	Service-authorized job claims and result submission.

File inventory: Domain models and schemas

Base path: src/withaq/

File	Responsibility
domain/___init___py	Domain package marker.
domain/enums.py	Trust, plan, root and dispatch states.
domain/errors.py	Feasibility, version and authorization failures.
domain/network.py	Device, Incident, Plan and action constraints.
domain/contracts.py	Function/task contracts and AND/OR dependencies.
domain/data.py	SourceRoot, Artifact, Run and Manifest.
domain/egress.py	Frozen payload, approval and dispatch request.
schemas/___init___py	Transport-schema package marker.
schemas/network.py	Pydantic network requests and snapshots.
schemas/data.py	Source, run and lineage payloads.
schemas/egress.py	Purpose, destination, account and hash fields.
schemas/jobs.py	Job-claim, heartbeat and result schemas.
schemas/common.py	UUIDs, versions and error envelope.

File inventory: Decision and guard services

Base path: src/withaq/services/

File	Responsibility
__init__.py	Domain-service package marker.
inventory.py	Register a device and enforcement-point mapping.
contracts.py	Approve contracts with owner and measurement evidence.
incidents.py	Assess suspect scope without blanket revocation.
snapshots.py	Freeze graphs, contracts and enforcement versions.
planner.py	MFSC enumeration, candidate ranking and deadlines.
validation.py	Request independent validation and bind its report.
containment.py	Authorization, application and reconciliation lifecycle.
lineage.py	Record inputs, root closure and reject lineage cycles.
guards.py	Check every root at read and publish admission.
revocation.py	Revocation-barrier transaction and outbox event.
recovery.py	Rebuild from alternatives, validate and promote.
disclosure.py	Inspect content, destination and purpose; sanitize.
admission.py	Lock roots and admit exact dispatch bytes.
jobs.py	Lease jobs, version attempts and reject stale results.
audit.py	Access-controlled audit events without raw content.

File inventory: Persistence and migrations

Base path: withaq/ (root)

File	Responsibility
src/withaq/repositories/__init__.py	Repository package marker.
src/withaq/repositories/models.py	ORM tables, keys and constraints.
src/withaq/repositories/network.py	Asset, contract and plan queries.
src/withaq/repositories/data.py	Roots, artifacts and lineage queries.
src/withaq/repositories/egress.py	Requests, approvals and dispatch queries.
src/withaq/repositories/jobs.py	Outbox, job row leasing and lock ordering.
migrations/env.py	Load metadata and DATABASE_URL from settings.
migrations/script.py.mako	Migration generation template.
migrations/versions/0001_network.py	Network, contract and decision tables.
migrations/versions/0002_lineage.py	Roots, artifacts, runs and incident links.
migrations/versions/0003_jobs_egress.py	Jobs, approvals, dispatch and audit tables.

File inventory: Adapters and separate processes

Base path: src/withaq/

File	Responsibility
adapters/__init__.py	Adapter package marker.
adapters/network.py	NetworkAdapter interface and restricted PEP client.
adapters/network_mock.py	Deterministic mock; never records real enforcement.
adapters/model.py	Model interface that cannot bypass disclosure guard.
adapters/model_mock.py	Fixed responses and controllable delay for revoke races.
adapters/storage.py	Local storage using bounded IDs outside web roots.
adapters/internal_api.py	Authenticated service client with timeouts and request IDs.
worker/__init__.py	Worker package marker.
worker/main.py	Claim loop, heartbeat and graceful stop.
worker/handlers.py	Planning, revoke, cleanup and recovery handlers.
worker/reconcile.py	Reconcile recorded intents with adapter state.
validator/__init__.py	Validator package marker.
validator/main.py	Separate internal validation service.
validator/checks.py	Recompute constraints without planner feasibility imports.
gateway/__init__.py	Dispatch-gateway package marker.
gateway/main.py	Accept admitted jobs and record transmission attempts.
gateway/provider.py	Approved provider adapter and unknown-outcome handling.
pep/__init__.py	Enforcement-agent package marker.
pep/main.py	Internal endpoint accepting only predefined plan actions.
pep/nft.py	Compile allowed actions into nftables rules.

File inventory: User interface

Base path: apps/console/

File	Responsibility
package.json	dev, build and test scripts and UI dependencies.
package-lock.json	Generated and committed frontend lock file.
index.html	React document entry point.
vite.config.ts	Development port and /v1 and /health proxies.
tsconfig.json	Project TypeScript configuration references.
tsconfig.app.json	Application TypeScript settings.
tsconfig.node.json	TypeScript settings for Node tooling.
eslint.config.js	Frontend lint rules.
src/main.tsx	Mount React and initialize language.
src/App.tsx	Route screens and handle application errors.
src/api/client.ts	Single HTTP client handling 409 and 403.
src/api/types.ts	Types generated or checked against OpenAPI.
src/pages/Devices.tsx	Device inventory, trust and enforcement capabilities.
src/pages/Contracts.tsx	Function contracts, bounds and owners.

File inventory: User interface | continued

File	Responsibility
src/pages/Incident.tsx	Incident reasons, plans and observed measurements.
src/pages/DataImpact.tsx	Roots, derivatives, holds and recovery.
src/pages/Disclosure.tsx	Disclosure request, frozen bytes and approval.
src/pages/Evidence.tsx	Role-scoped evidence trace and export.
src/components/StatusBadge.tsx	Textual states, not color-only indicators.
src/components/DependencyGraph.tsx	Graph display without authoritative security logic.
src/components/ContractPanel.tsx	Per-contract observations and violations.
src/i18n/ar.json	Arabic labels and RTL support.
src/i18n/en.json	Equivalent English labels.
src/styles.css	Color, spacing, typography and direction.
playwright.config.ts	Browser-test environment configuration.
tests/incident.spec.ts	Incident and infeasible-plan user journeys.
tests/revocation.spec.ts	Late-result rejection and alternative recovery.

File inventory: Lab data and policy

Base path: withaq/ (root)

File	Responsibility
lab/topology.yml	Isolated lab addresses, ports and links.
lab/contracts.yml	FC1, FC2 and FC3 measurement limits.
lab/nftables/withaq.nft	Rules template for the lab namespace only.
lab/generator.py	Sequenced synthetic samples and alerts.
lab/services.py	Monitor, alert receiver and synthetic protected target.
lab/probes.py	Probe contracts and denial of new/existing flows.
lab/control.py	up, down and status for owned lab resources only.
fixtures/network.json	Known network graph and action fixture.
fixtures/data.json	Synthetic S1, S2, S3 and derivative fixtures.
fixtures/disclosures.json	Allow, block, sanitize and approval cases.
policies/disclosure.yml	Versioned destination, account and purpose policy.
policies/roles.yml	Role-to-command and read permissions.
policies/retention.yml	Separate evidence, content and backup retention.

File inventory: Tests and engineering records

Base path: withaq/ (root)

File	Responsibility
tests/conftest.py	Test DB, identity, mock server and controlled clock.
tests/unit/test_planner.py	Best plan, empty plan, infeasibility and timeout.
tests/unit/test_contracts.py	AND/OR dependencies and independent alert-loss checks.
tests/unit/test_validator.py	Reject defective plans using independent oracle cases.
tests/unit/test_lineage.py	Complete root closure, cycles and unknown lineage.
tests/unit/test_disclosure.py	Arabic/English inspection and sanitized reinspection.
tests/integration/test_migrations.py	Migrations on an empty DB and key constraints.
tests/integration/test_revoke_race.py	Revocation racing with read and publication.
tests/integration/test_egress_race.py	Pre/post-admission revocation and payload mutation.
tests/integration/test_job_restart.py	Worker crash and stale-attempt result rejection.
tests/integration/test_restore.py	Isolated restore and revocation-ledger replay.
tests/security/test_access.py	User/service permissions and API bypass checks.
tests/security/test_bypass.py	Block direct provider access in lab deployment.
tests/lab/test_enforcement.py	Actual ruleset, session and contract verification.
experiments/run_matrix.py	Run comparative configurations with recorded seeds.
experiments/analyze.py	Analyze raw results without excluding failures.
experiments/manifest.json	Software, hardware, policy and seed manifest.
docs/architecture.md	Maintain architecture decisions and trust boundaries.
docs/runbook.md	Start, stop and restore procedures.
docs/adr/001-state-authority.md	State authority and guard decision record.
docs/adr/002-network-modes.md	Separation of mock and lab modes.
docs/acceptance.md	Evidence required at each delivery gate.

Delivery order and definition of done

Assign work around modules with contracts and tests, not only UI screens. A domain owner builds contracts/planning; a state owner builds lineage/guards; an operations owner builds adapters/lab; a console/evaluation owner assembles and tests the journey. One person can fill multiple roles, but planner review should remain distinct from validator testing.

Delivery	Required completion evidence
D1	Reproducible environment, health service, migrated database and identity.
D2	Planner, validator, empty plan, infeasibility, timeout and oracle cases.
D3	Revocation guards, complete lineage, late-result race and alternative recovery.
D4	Disclosure gateway, exact-byte approval and unknown-send handling.
D5	Arabic/English console and complete mock scenario.
D6	Real lab enforcement, session/contract probes and bypass tests.
D7	Repeatable operation, backup/restore, raw results and documented limits.

The prototype is ready for a technical demonstration when another developer can recreate it from Git, locks and fixtures, run the scenario and collect equivalent evidence. This does not establish hospital readiness, regulatory approval or scientific priority. WITHAQ remains a contract-constrained response across devices, data and disclosure, explicitly reporting uncertainty when evidence is missing.

Glossary

Term	Definition in WITHAQ
IoT / IoMT / OT	Connected devices / connected medical devices / operational technology; the lab models monitoring only.
MFSC	Minimum restriction set within specified actions and model under hard critical contracts.
PEP	Point enforcing policy on actual traffic.
Contract	Measurable function or task specification with owner and version.
Manifest	Sealed record of every input to one run, its roots and versions.
Lineage	Origin and derivation relations, not text similarity or model inference.
Root / Artifact	Authorization-bearing origin / immutable ingested or derived content version.
Barrier / Epoch	Binding ordering point / increasing version used to detect stale decisions.
Outbox	Job event saved in the same state transaction so it can be resumed.
D / C	Proven derivative set / broader precautionary hold scope.
ADMITTED	Dispatch request that crossed the authorization barrier, not proof of provider receipt.
UNKNOWN	A judgment unsupported by sufficient current evidence; neither safe nor infeasible.

References and implementation sources

Design references are retained from the preceding report; official setup sources were reviewed on 28 September 2026. These references do not establish WITHAQ results or certification. Earlier team documents establish the concept; this edition adds a proposed implementation specification.

Debenedetti, E. et al. (2025). Defeating Prompt Injections by Design. arXiv:2503.18813, v2.

<https://arxiv.org/abs/2503.18813>

Lear, E., Droms, R. and Romascanu, D. (2019). Manufacturer Usage Description Specification. RFC 8520.

<https://www.rfc-editor.org/rfc/rfc8520.html>

Moreau, L. and Missier, P. (eds.) (2013). PROV-DM: The PROV Data Model. W3C Recommendation, 30 April.

<https://www.w3.org/TR/prov-dm/>

Netfilter Project (n.d.). The nftables project. Official project documentation.

<https://netfilter.org/projects/nftables/index.html>

Object Management Group (2017). Unified Modeling Language, Version 2.5.1.

<https://www.omg.org/spec/UML/2.5.1>

Peffer, K. et al. (2006). The Design Science Research Process: A Model for Producing and Presenting Information Systems Research. DESRIST; repository copy deposited 2020.

<https://arxiv.org/abs/2006.02763>

Rose, S., Borchert, O., Mitchell, S. and Connelly, S. (2020). Zero Trust Architecture. NIST SP 800-207.

<https://csrc.nist.gov/pubs/sp/800/207/final>

Stouffer, K. et al. (2023). Guide to Operational Technology (OT) Security. NIST SP 800-82 Rev. 3.

<https://csrc.nist.gov/pubs/sp/800/82/r3/final>

Official setup links and edition control

Docker (n.d.). Install Docker Engine on Ubuntu.

<https://docs.docker.com/engine/install/ubuntu/>

Docker (n.d.). Control startup and shutdown order in Compose.

<https://docs.docker.com/compose/how-tos/startup-order/>

FastAPI (n.d.). First Steps.

<https://fastapi.tiangolo.com/tutorial/first-steps/>

Python Software Foundation (n.d.). venv: Creation of virtual environments.

<https://docs.python.org/3/library/venv.html>

Vite (n.d.). Getting Started.

<https://vite.dev/guide/>

Alembic (n.d.). Tutorial.

<https://alembic.sqlalchemy.org/en/latest/tutorial.html>

PostgreSQL Global Development Group (n.d.). Explicit Locking.

<https://www.postgresql.org/docs/current/explicit-locking.html>

WITHAQ | Edition 2.0

28 September 2026. Equivalent editions covering concept, architecture, build/run procedures and file inventory. The complete application was not built or run during this reporting task; the deliverables are the two documents, layout verification and the illustrative decision computation.